

Arquitectura de Software

¿Qué es Arquitectura de Software?

Es el conjunto de decisiones fundamentales sobre la organización y estructura de un sistema informático. Establece los componentes principales (módulos, capas, servicios), cómo interactúan entre sí y las restricciones bajo las cuales deben operar. Su objetivo principal es asegurar que el software cumpla tanto con las funciones requeridas como con los atributos de calidad (escalabilidad, rendimiento, seguridad, disponibilidad y facilidad de mantenimiento).

Principios SOLID

Es un acrónimo introducido por Robert C. Martin ("Uncle Bob") que agrupa cinco principios de diseño para la programación orientada a objetos. Su propósito es reducir el acoplamiento, aumentar la cohesión y facilitar la evolución del código sin romper funcionalidades existentes.

- **S — Single Responsibility Principle (Principio de Responsabilidad Única):**
 - ❖ **Concepto:** Una clase debe tener una sola razón para cambiar, lo que significa que debe encargarse de una única tarea o responsabilidad dentro del dominio del problema.
 - ❖ **Objetivo:** Evitar clases masivas ("God objects") que mezclen lógica de negocio, persistencia y presentación.
- **O — Open/Closed Principle (Principio de Abierto/Cerrado):**
 - ❖ **Concepto:** Las entidades de software (clases, módulos, funciones) deben estar abiertas para su extensión, pero cerradas para su modificación.
 - ❖ **Objetivo:** Permitir agregar nuevos comportamientos o requerimientos mediante herencia o polimorfismo sin necesidad de alterar el código fuente original ya probado.
- **L — Liskov Substitution Principle (Principio de Sustitución de Liskov):**
 - ❖ **Concepto:** Si una clase *B* es subtipo de una clase *A*, los objetos de tipo *A* deben poder sustituirse por objetos de tipo *B* sin alterar el funcionamiento correcto del programa.

- ❖ **Objetivo:** Garantizar que las clases hijas respeten estrictamente los contratos y comportamientos esperados de la clase base.
- **I — Interface Segregation Principle (Principio de Segregación de Interfaces):**
 - ❖ **Concepto:** Los clientes no deben estar forzados a depender de interfaces o métodos que no utilizan. Es preferible tener muchas interfaces específicas y pequeñas que una sola interfaz general y sobrecargada.
 - ❖ **Objetivo:** Desacoplar sistemas evitando que los módulos implementen métodos vacíos o innecesarios.
- **D — Dependency Inversion Principle (Principio de Inversión de Dependencias):**
 - ❖ **Concepto:** Los módulos de alto nivel (la lógica central de negocio) no deben depender de módulos de bajo nivel (bases de datos, controladores de hardware, llamadas de red); ambos deben depender de abstracciones (interfaces). Además, las abstracciones no deben depender de los detalles; los detalles deben depender de las abstracciones.
 - ❖ **Objetivo:** Reducir la dependencia directa de tecnologías o implementaciones concretas, facilitando pruebas unitarias y cambios de infraestructura.

APLICACIÓN DE SOLID AL SISTEMA DE GESTIÓN DE MEGATLON

Los 5 Principios SOLID aplicados a Megatlón

- **S (Single Responsibility - Responsabilidad Única):**
 - **Problema común:** Que una sola clase ClienteController o AccesoService valide el CI, descuente cupos de la base de datos, calcule deudas de cuotas y además dibuje la pantalla del monitor.
 - **Solución en Megatlón:** ControlAccesoService solo valida si la persona puede pasar; si la respuesta es afirmativa, le delega a NotificadorSaldoService la tarea de proyectar en el monitor: "*Bienvenido, te quedan X ingresos*". Cada clase tiene una sola razón para cambiar.

- **O (Open/Closed - Abierto a la extensión, cerrado a la modificación):**
 - **Problema común:** Usar un switch o varios if para calcular precios según el tipo de plan. Si el dueño crea una nueva disciplina o paquete mixto, tendrías que modificar el código central y arriesgarte a romper los cobros existentes.
 - **Solución en Megatlón:** Creas una clase abstracta o interfaz Plan. Si mañana Megatlón lanza el "Plan Trimestral CrossFit", simplemente creas una clase nueva PlanCrossfit que hereda de Plan. Extiendes el sistema sin tocar ni una sola línea del código que ya funciona.
- **L (Liskov Substitution - Sustitución de Liskov):**
 - **Problema común:** Tener una clase padre y que una clase hija no pueda cumplir con lo prometido (por ejemplo, lanzar una excepción de "método no implementado").
 - **Solución en Megatlón:** Tanto PlanMensual como PlanSesion heredan de Plan. El sistema de cobros puede invocar plan.calcularMontoCobro() sobre cualquier instancia sin saber cuál es cuál, y el sistema funcionará correctamente sin excepciones inesperadas.
- **I (Interface Segregation - Segregación de Interfaces):**
 - **Problema común:** Una interfaz sobrecargada con métodos de login web, cobros de planes, tiqueo de puerta y edición de sueldos.
 - **Solución en Megatlón:** La dividimos en contratos específicos. Por ejemplo, IAccesoTerminal solo expone tiquearIngreso(ci). La pantalla física de entrada o el torniquete solo interactúa con esa acción puntual, sin enterarse de contraseñas ni salarios de empleados.
- **D (Dependency Inversion - Inversión de Dependencias):**
 - **Problema común:** Que ControlAccesoService instancie directamente un conector a MySQL (new MySQLConnection()). Si la sucursal pierde internet y necesita usar una base de datos local ligera (SQLite o caché en memoria) para operar offline, el sistema colapsaría.
 - **Solución en Megatlón:** El servicio de acceso depende de una abstracción llamada IClienteRepository. En internet normal usa

ClienteRepositoryMySQL, y si la red cae, el sistema puede cambiar a un repositorio local sin modificar el servicio de acceso.

Explicación del código mermaid

1. Las Clases y sus Responsabilidades (S - Single Responsibility)

Fragmento de código

```
class ControlAccesoService {  
    -IClientRepository clienteRepo  
    +validarIngreso(ci: string, sucursalId: int) bool  
}  
  
class NotificadorSaldoService {  
    +mostrarSaldoPantalla(ci: string, saldoRestante: int) void  
}
```

- **ControlAccesoService:** Su único trabajo es tomar decisiones de negocio (¿la persona pagó? ¿puede pasar?). Tiene el método validarIngreso().
- **NotificadorSaldoService:** Su único trabajo es pintar el mensaje en la pantalla física del gimnasio (mostrarSaldoPantalla()).
- **Por qué cumple la S:** Si mañana el dueño pide cambiar el diseño de la alerta en pantalla, solo modificas NotificadorSaldoService y no tocas para nada la lógica de acceso del gimnasio.

2. Las Interfaces Pequeñas (I - Interface Segregation)

Fragmento de código

```
class IAutenticable {  
    <<interface>>  
    +autenticar(ci: string, password: string) bool  
}  
  
class IAccesoTerminal {  
    <<interface>>
```

```
+tikearIngreso(ci: string) bool
}
```

- Las etiquetas `<<interface>>` avisan que no son clases con código, sino **contratos o listas de reglas**.
- **IAutenticable**: Define la regla de "saber iniciar sesión con usuario y contraseña" (usada para la web o la app).
- **IAccesoTerminal**: Define la regla exclusiva de "saber tikear en la puerta".
- **Por qué cumple la I**: En vez de obligar a que todo el mundo use una sola interfaz gigante, las separamos para que cada dispositivo use solo lo que le importa.

3. La Abstracción de Datos (D - Dependency Inversion)

Fragmento de código

```
class IClienteRepository {
    <<interface>>

    +obtenerPorCI(ci: string) Cliente
    +descontarCupo(ci: string) void
}

class ClienteRepositoryMySQL {
    +obtenerPorCI(ci: string) Cliente
    +descontarCupo(ci: string) void
}
```

- **IClienteRepository**: Es un contrato que dice: *"Quien quiera ser repositorio debe saber buscar por CI y descontar cupos"*. No le importa qué base de datos uses.
- **ClienteRepositoryMySQL**: Es la implementación real que se conecta a MySQL.
- **Por qué cumple la D**: El servicio de acceso depende del contrato general (IClienteRepository), no de la base de datos directa. Si mañana cambian MySQL por PostgreSQL o un almacenamiento local offline para la sucursal, el servicio de acceso ni se entera.

4. La Herencia de Planes (O - Open/Closed y L - Liskov Substitution)

Fragmento de código

```
class Plan {  
    <<abstract>>  
  
    +nombre: string  
  
    +precioBase: float  
  
    +calcularMontoCobro()* float  
}  
  
class PlanMensual {  
  
    +meses: int  
  
    +calcularMontoCobro() float  
}  
  
class PlanSesion {  
  
    +cantidadSesiones: int  
  
    +calcularMontoCobro() float  
}
```

- **Plan:** Es una clase abstracta (un molde base) con un método general `calcularMontoCobro()`.
- **PlanMensual y PlanSesion:** Son clases hijas que heredan de Plan y calculan su cobro a su manera.
- **Por qué cumple la O y la L:**
 - **O (Abierto/Cerrado):** Si Megatlón inventa un "Plan Anual", creas una clase nueva `PlanAnual` sin modificar ni romper las anteriores.
 - **L (Liskov):** El sistema puede procesar cualquier cobro usando la clase padre Plan, y tanto `PlanMensual` como `PlanSesion` responderán correctamente sin fallar.